

ROTEIRO DE LABORATÓRIO 6

A primeira função da equipe, ligada ao site

Aula 7, 1º de outubro de 2026 · AWS Lambda pela interface ou pelo CloudShell

O que você vai construir

Na Aula 6 a equipe publicou o site estático no S3. Hoje ele ganha uma parte que "pensa": uma função AWS Lambda, escrita em Python, que responde com uma mensagem e o horário de Caruaru. O site chama a função pelo JavaScript e mostra a resposta.

Como no Roteiro Completo, há dois caminhos que chegam ao mesmo resultado: o Caminho A, pela interface do console (passos 1 a 7), e o Caminho B, pelo terminal do CloudShell (passos 8 a 12). A Parte 3, com o custo e a limpeza, é para todos.

AO FINAL DESTES ROTEIRO VOCÊ TERÁ

- Uma função Lambda em Python, testada no console.
- Uma URL pública da função, com CORS liberado para o site chamá-la.
- O site da equipe no S3 mostrando a resposta da função.
- Os logs de cada execução no CloudWatch.
- Uma linha nova no relatório de custo, com a estimativa da Lambda.

Antes de começar

Confira estes itens antes de começar.

- Conta AWS com o usuário do dia a dia e o alerta de orçamento (Parte 1 do Roteiro Completo).
- O site da equipe no ar pelo S3 (Roteiro Completo, passos 11 e 12), e o nome do bucket anotado.
- Região N. Virginia (us-east-1) selecionada no canto superior direito do console.
- Troque "Equipe X" pelo nome da sua equipe sempre que ele aparecer.

Passo a passo

Leia o "Por que isso importa" antes de clicar e só avance quando o "Confira" estiver certo. O esquema abaixo mostra onde a função entra no projeto.

ONDE A FUNÇÃO ENTRA NO PROJETO

O site continua no S3. O JavaScript da página chama a URL da função; a função responde em JSON; o site mostra a resposta.

POR QUE ISSO IMPORTA

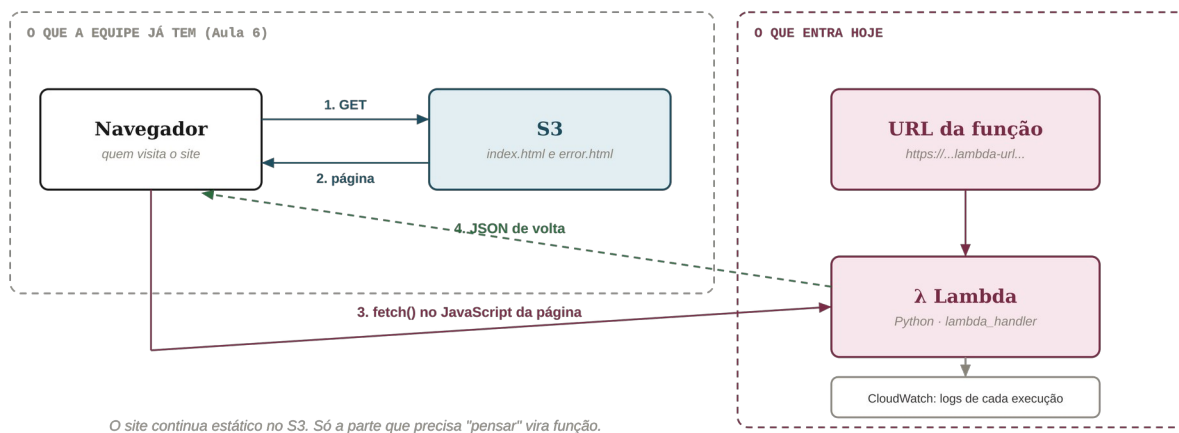
Caminho A: passos 1 a 7, clicando no console. Caminho B: passos 8 a 12, no CloudShell. Escolha um. A Parte 3 (passo 13) é para todos.

CAMINHO A · PELA INTERFACE, NO CONSOLE (passos 1 a 7)

Se escolheu o terminal, pule para o passo 8.

1 Caminho A · Criar a função

- Na busca do topo do console, digite Lambda e abra o serviço.
- Clique em Create function e deixe marcado Author from scratch.
- Function name: status-equipeX (só letras, números e hífen).
- Runtime: Python 3.13 (ou a versão de Python mais recente da lista). Architecture: x86_64.
- Em Permissions, deixe Create a new role with basic Lambda permissions e clique em Create function.



O que a equipe já tem (Aula 6) e o que entra hoje.

POR QUE ISSO IMPORTA

A opção de permissões cria um papel (role) que só deixa a função escrever logs. É o princípio do menor privilégio da Aula 2: a função não pode mexer em mais nada da conta.

CONFIRA ANTES DE SEGUIR

Abre a página da função status-equipeX, com o editor de código embaixo e o arquivo `lambda_function.py`.

2 Caminho A · Escrever o código e publicar

- No editor, apague o código de exemplo de `lambda_function.py` e escreva o código abaixo, trocando Equipe X pelo nome da equipe.
- Clique em Deploy. Sem o Deploy, a função continua rodando o código antigo.

```
import json
from datetime import datetime, timezone, timedelta

def lambda_handler(event, context):
    agora = datetime.now(timezone(timedelta(hours=-3)))
    return {
        "statusCode": 200,
        "headers": {"Content-Type": "application/json"},
        "body": json.dumps({
            "mensagem": "Olá da Lambda da Equipe X!",
            "horario": agora.strftime("%H:%M"),
        })
    }
```

POR QUE ISSO IMPORTA

O `lambda_handler` é a porta de entrada: a Lambda chama essa função a cada evento. O dicionário devolvido vira a resposta HTTP: `statusCode` é o código, `headers` são os cabeçalhos e `body` é o texto.

CONFIRA ANTES DE SEGUIR

Aparece a mensagem verde "Successfully updated the function status-equipeX".

3 Caminho A · Testar no console

- Clique em Test. Na primeira vez, ele pede um evento de teste: Event name: teste, deixe o JSON de exemplo e clique em Save.
- Clique em Test de novo e leia o resultado em Execution results.

POR QUE ISSO IMPORTA

O teste simula um evento sem precisar de navegador. Repare em `Duration` e `Billed Duration`: é por esse tempo que a Lambda cobra.

ATENÇÃO

Erro de sintaxe ou de indentação? A mensagem aponta a linha. Corrija, clique em Deploy e teste de novo.

CONFIRA ANTES DE SEGUIR

O Response mostra statusCode 200 e um body com a mensagem e o horário.

4 Caminho A · Criar a URL da função

- Na página da função, abra a aba Configuration e, no menu da esquerda, Function URL.
- Clique em Create function URL. Auth type: NONE.
- Abra Additional settings, marque Configure cross-origin resource sharing (CORS) e deixe Allow origin como *. Em Allow methods, marque GET.
- Clique em Save e copie a Function URL que aparece.

POR QUE ISSO IMPORTA

NONE quer dizer que qualquer pessoa com o endereço pode chamar a função. Para um site público, é o esperado. O CORS autoriza o JavaScript de outro endereço, o do S3, a ler a resposta.

ATENÇÃO

Cada chamada conta no custo. Não publique a URL em grupos ou redes: ela só precisa estar no index.html do site.

CONFIRA ANTES DE SEGUIR

Abrindo a Function URL numa aba nova do navegador, aparece o JSON com a mensagem e o horário.

5 Caminho A · Ligar a função ao site

- No computador, abra o index.html do site da equipe e cole o trecho abaixo antes de </body>, trocando COLE_AQUI_A_URL_DA_FUNCAO pela URL copiada.
- No console, abra o S3, entre no bucket do site e faça Upload do index.html, substituindo o antigo.

```
<!-- cole antes de </body> no index.html -->
<p id="lambda">Chamando a função...</p>
<script>
  const URL_FUNCAO = "COLE_AQUI_A_URL_DA_FUNCAO";
  fetch(URL_FUNCAO)
    .then((r) => r.json())
    .then((d) => {
      document.getElementById("lambda").textContent =
        d.mensagem + " Agora são " + d.horario + ".";
    })
    .catch(() => {
      document.getElementById("lambda").textContent =
        "A função não respondeu.";
    });
</script>
```

POR QUE ISSO IMPORTA

O site continua estático no S3. Quem busca a informação é o navegador de quem visita: o fetch chama a URL e o JavaScript escreve a resposta na página.

CONFIRA ANTES DE SEGUIR

Abrindo o endereço do site do S3 (com http://), aparece a frase "Olá da Lambda da Equipe X! Agora são 19:42." com o horário de agora.

6 Caminho A · Ver os logs

- Na função, abra a aba Monitor e clique em View CloudWatch logs.
- Abra o log stream mais recente.

POR QUE ISSO IMPORTA

Cada execução gera linhas START, END e REPORT, com a duração e a memória usada. É aqui que se descobre o que deu errado quando a função falha.

CONFIRA ANTES DE SEGUIR

Há uma linha REPORT para cada vez que você abriu o site ou a URL.

7 Caminho A · Recarregar e comparar

- Recarregue o site três vezes seguidas e volte aos logs.
- Compare a Duration da primeira chamada do dia com a das seguintes.

1ª chamada
início a frio

prepara o ambiente

carrega o código

executa

2ª chamada
segundos depois

executa

o ambiente ainda está quente: resposta rápida

Depois de um
tempo parada

prepara o ambiente

carrega o código

executa

Por que importa

O primeiro usuário espera mais. Para um formulário de contato, tanto faz. Para um pagamento com tempo limite, pode fazer diferença.

Início a frio (cold start): a AWS precisa preparar um ambiente novo antes de rodar a sua função pela primeira vez.

O início a frio aparece nos logs como Init Duration, só na primeira chamada.

CONFIRA ANTES DE SEGUIR

A primeira linha REPORT tem Init Duration; as seguintes, não. É o início a frio da aula, medido na sua função. Siga para a Parte 3 (passo 13).

CAMINHO B · PELO TERMINAL, NO CLOUDSHELL (passos 8 a 12)

Se você fez o Caminho A, este é opcional. Faça tudo na mesma janela do CloudShell: os comandos guardam valores em variáveis, como \$ROLE e \$URL.

8 Caminho B · Escrever a função

- Abra o CloudShell pelo ícone >_ na barra do topo e crie o arquivo com o nano:

```
nano lambda_function.py
```

POR QUE ISSO IMPORTA

Escreva o mesmo código do passo 2, trocando Equipe X pelo nome da equipe, salve com Ctrl+O e Enter, e saia com Ctrl+X. Depois compacte o arquivo:

```
zip funcao.zip lambda_function.py
```

CONFIRA ANTES DE SEGUIR

O comando ls mostra lambda_function.py e funcao.zip.

9 Caminho B · Criar o papel da função

- A função precisa de um papel (role) com permissão para escrever logs. Rode:

```
cat > confianca.json <<FIM
{"Version": "2012-10-17", "Statement": [{"Effect": "Allow",
  "Principal": {"Service": "lambda.amazonaws.com"},
  "Action": "sts:AssumeRole"}]}
FIM
aws iam create-role --role-name lambda-equipeX \
  --assume-role-policy-document file://confianca.json
aws iam attach-role-policy --role-name lambda-equipeX --policy-arn \
  arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
ROLE=$(aws iam get-role --role-name lambda-equipeX \
  --query Role.Arn --output text)
sleep 10
```

POR QUE ISSO IMPORTA

O primeiro arquivo diz quem pode usar o papel: o serviço Lambda. A política anexada só permite escrever logs. O sleep dá tempo para o IAM espalhar o papel novo.

CONFIRA ANTES DE SEGUIR

echo \$ROLE mostra um endereço que começa com arn:aws:iam:: e termina com role/lambda-equipeX.

10 Caminho B · Criar e testar a função

- Rode:

```
aws lambda create-function --function-name status-equipeX \  
  --runtime python3.13 --handler lambda_function.lambda_handler \  
  --role $ROLE --zip-file fileb://funcao.zip  
aws lambda wait function-active-v2 --function-name status-equipeX  
aws lambda invoke --function-name status-equipeX resposta.json  
cat resposta.json
```

ATENÇÃO

Erro "The role defined for the function cannot be assumed by Lambda"? O papel ainda não se espalhou: espere uns segundos e rode o create-function de novo.

CONFIRA ANTES DE SEGUIR

O invoke mostra "StatusCode": 200 e o cat mostra o body com a mensagem e o horário.

11 Caminho B · Criar a URL e liberar o acesso

- Rode:

```
aws lambda create-function-url-config --function-name status-equipeX \
  --auth-type NONE --cors '{"AllowOrigins": ["*"], "AllowMethods": ["GET"]}'
aws lambda add-permission --function-name status-equipeX \
  --statement-id url-publica --action lambda:InvokeFunctionUrl \
  --principal "*" --function-url-auth-type NONE
aws lambda add-permission --function-name status-equipeX \
  --statement-id invoke-pela-url --action lambda:InvokeFunction \
  --principal "*" --invoked-via-function-url
URL=$(aws lambda get-function-url-config --function-name status-equipeX \
  --query FunctionUrl --output text)
curl $URL
```

POR QUE ISSO IMPORTA

Pelo terminal, a permissão pública não é criada sozinha como no console: são duas regras, uma para chamar pela URL e outra para executar a função, e a segunda só vale quando a chamada vem pela URL.

ATENÇÃO

Deu 403 Forbidden no curl? Faltou uma das duas regras add-permission.

CONFIRA ANTES DE SEGUIR

O curl mostra o JSON com a mensagem e o horário. Anote a URL com echo \$URL.

12 Caminho B · Ligar ao site no S3

- Troque NOME-DO-BUCKET pelo bucket do site da equipe e rode:

```
B=NOME-DO-BUCKET
aws s3 cp s3://$B/index.html index.html
nano index.html
```

POR QUE ISSO IMPORTA

No nano, cole antes de `</body>` o trecho de JavaScript do passo 5, com a URL do echo `$URL` no lugar de `COLE_AQUI_A_URL_DA_FUNCAO`. Salve e saia, e envie de volta:

```
aws s3 cp index.html s3://$B/index.html
echo "http://$B.s3-website-us-east-1.amazonaws.com"
```

CONFIRA ANTES DE SEGUIR

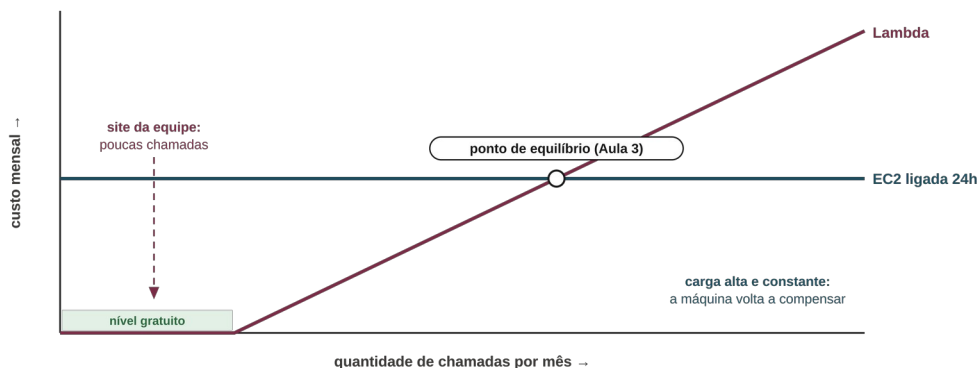
O endereço do site mostra a frase com a mensagem da função e o horário de agora.

PARTE 3 · CUSTO E LIMPEZA (para todos)

Os dois caminhos terminam aqui.

13 Custo, relatório e quando apagar

- Estime quantas vezes por mês o site será aberto. Calcule: chamadas $\times$ 0,125 GB $\times$ duração em segundos (veja a Duration nos logs).
- Compare com o nível gratuito: 1 milhão de chamadas e 400 mil GB-segundo por mês.
- Acrescente uma linha ao relatório de custo da entrega: "Lambda: N chamadas por mês, X GB-segundo, custo estimado US\$ 0,00 (nível gratuito)".



Com poucas chamadas, a Lambda fica no nível gratuito; carga alta e constante volta a favorecer a máquina.

ATENÇÃO

Mantenha a função até a aula de 15/10: ela vira a base da API. Depois da avaliação, apague a função em Actions > Delete (ou `aws lambda delete-function --function-name status-equipeX`) e o papel `lambda-equipeX` no IAM.

CONFIRA ANTES DE SEGUIR

O relatório de custo tem a linha da Lambda, com o cálculo e o número de chamadas usado na estimativa.

Se sobrar tempo

Faça a função ler um parâmetro da URL. Em Python, `event.get("queryStringParameters")` traz o que vem depois do `?` no endereço: com `?nome=Ana`, a mensagem pode responder "Olá, Ana!". Teste abrindo a URL da função com `?nome=SeuNome` no final.

Pergunta para levar para 15/10: e se a equipe tiver dez funções, cada uma com a sua URL? É o problema que o API Gateway resolve.